

Objektinis\_programavimas\_2

Generated by Doxygen 1.13.2



|                                                |           |
|------------------------------------------------|-----------|
| <b>1 README</b>                                | <b>1</b>  |
| <b>2 Hierarchical Index</b>                    | <b>7</b>  |
| 2.1 Class Hierarchy                            | 7         |
| <b>3 Class Index</b>                           | <b>9</b>  |
| 3.1 Class List                                 | 9         |
| <b>4 File Index</b>                            | <b>11</b> |
| 4.1 File List                                  | 11        |
| <b>5 Class Documentation</b>                   | <b>13</b> |
| 5.1 Studentas Class Reference                  | 13        |
| 5.1.1 Constructor & Destructor Documentation   | 14        |
| 5.1.1.1 Studentas() [1/6]                      | 14        |
| 5.1.1.2 Studentas() [2/6]                      | 14        |
| 5.1.1.3 Studentas() [3/6]                      | 14        |
| 5.1.1.4 Studentas() [4/6]                      | 14        |
| 5.1.1.5 Studentas() [5/6]                      | 15        |
| 5.1.1.6 Studentas() [6/6]                      | 15        |
| 5.1.1.7 ~Studentas()                           | 15        |
| 5.1.2 Member Function Documentation            | 15        |
| 5.1.2.1 clearStud()                            | 15        |
| 5.1.2.2 egzaminas()                            | 15        |
| 5.1.2.3 galutinisMed()                         | 15        |
| 5.1.2.4 galutinisVid()                         | 15        |
| 5.1.2.5 isClear()                              | 15        |
| 5.1.2.6 operator"!="()                         | 15        |
| 5.1.2.7 operator=() [1/2]                      | 16        |
| 5.1.2.8 operator=() [2/2]                      | 16        |
| 5.1.2.9 operator==()                           | 16        |
| 5.1.2.10 pazymiai()                            | 16        |
| 5.1.2.11 printInfo()                           | 16        |
| 5.1.2.12 setEgzaminas()                        | 16        |
| 5.1.2.13 setGalutinisM()                       | 16        |
| 5.1.2.14 setGalutinisV()                       | 16        |
| 5.1.2.15 setPazymiai()                         | 16        |
| 5.1.2.16 skaiciuotiGalutiniSuMed()             | 17        |
| 5.1.2.17 skaiciuotiGalutiniSuVid()             | 17        |
| 5.1.3 Friends And Related Symbol Documentation | 17        |
| 5.1.3.1 operator<<                             | 17        |
| 5.1.3.2 operator>>                             | 17        |
| 5.2 Timer Class Reference                      | 17        |
| 5.2.1 Constructor & Destructor Documentation   | 17        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 5.2.1.1 Timer()                                         | 17        |
| 5.2.2 Member Function Documentation                     | 18        |
| 5.2.2.1 elapsed()                                       | 18        |
| 5.2.2.2 reset()                                         | 18        |
| 5.3 Zmogus Class Reference                              | 18        |
| 5.3.1 Constructor & Destructor Documentation            | 18        |
| 5.3.1.1 Zmogus() [1/2]                                  | 18        |
| 5.3.1.2 Zmogus() [2/2]                                  | 19        |
| 5.3.1.3 ~Zmogus()                                       | 19        |
| 5.3.2 Member Function Documentation                     | 19        |
| 5.3.2.1 pavarde()                                       | 19        |
| 5.3.2.2 printInfo()                                     | 19        |
| 5.3.2.3 setPavarde()                                    | 19        |
| 5.3.2.4 setVardas()                                     | 19        |
| 5.3.2.5 vardas()                                        | 19        |
| 5.3.3 Member Data Documentation                         | 19        |
| 5.3.3.1 pavarde_                                        | 19        |
| 5.3.3.2 vardas_                                         | 19        |
| <b>6 File Documentation</b>                             | <b>21</b> |
| 6.1 1_strategija/konteineriuTyrimas.cpp File Reference  | 21        |
| 6.1.1 Function Documentation                            | 21        |
| 6.1.1.1 main()                                          | 21        |
| 6.2 1_strategija/strategija1.cpp File Reference         | 21        |
| 6.2.1 Function Documentation                            | 21        |
| 6.2.1.1 main()                                          | 21        |
| 6.3 1_strategija/strategija2.cpp File Reference         | 22        |
| 6.3.1 Function Documentation                            | 22        |
| 6.3.1.1 main()                                          | 22        |
| 6.4 1_strategija/strategija3.cpp File Reference         | 22        |
| 6.4.1 Function Documentation                            | 22        |
| 6.4.1.1 main()                                          | 22        |
| 6.5 1_strategija/strategijos_bibl.h File Reference      | 22        |
| 6.5.1 Function Documentation                            | 23        |
| 6.5.1.1 nuskaitytiFailaT()                              | 23        |
| 6.5.1.2 rikiuotiPagalGalutiniMedT()                     | 23        |
| 6.5.1.3 rikiuotiPagalGalutiniVidT()                     | 23        |
| 6.5.1.4 rikiuotiPagalPavardeT()                         | 23        |
| 6.5.1.5 rikiuotiPagalVardaT()                           | 24        |
| 6.5.1.6 skirstytiStudentusSuTaisPaciaisKonteineriaisT() | 24        |
| 6.5.1.7 skirstytiStudentusSuVienuKonteineriuT()         | 24        |
| 6.5.1.8 skirstytiStudentusT()                           | 24        |

|                                           |    |
|-------------------------------------------|----|
| 6.5.1.9 studentuSkirstymas3()             | 24 |
| 6.6 strategijos_bibl.h                    | 24 |
| 6.7 catchTests.cpp File Reference         | 27 |
| 6.7.1 Macro Definition Documentation      | 27 |
| 6.7.1.1 CATCH_CONFIG_MAIN                 | 27 |
| 6.7.2 Function Documentation              | 28 |
| 6.7.2.1 TEST_CASE()                       | 28 |
| 6.8 humanClass.h File Reference           | 28 |
| 6.9 humanClass.h                          | 28 |
| 6.10 programa.cpp File Reference          | 28 |
| 6.10.1 Function Documentation             | 29 |
| 6.10.1.1 main()                           | 29 |
| 6.11 programaFunkcijos.cpp File Reference | 29 |
| 6.11.1 Function Documentation             | 29 |
| 6.11.1.1 generuotiFailus()                | 29 |
| 6.11.1.2 generuotiPazymius()              | 29 |
| 6.11.1.3 generuotiStudentus()             | 29 |
| 6.11.1.4 isvestiDuFailus()                | 30 |
| 6.11.1.5 nuskaitytiFaila()                | 30 |
| 6.11.1.6 operator<<()                     | 30 |
| 6.11.1.7 operator>>()                     | 30 |
| 6.11.1.8 pasirinktiFaila()                | 30 |
| 6.11.1.9 rikiuotiPagalGalutiniMed()       | 30 |
| 6.11.1.10 rikiuotiPagalGalutiniVid()      | 30 |
| 6.11.1.11 rikiuotiPagalPavarde()          | 30 |
| 6.11.1.12 rikiuotiPagalVarda()            | 31 |
| 6.11.1.13 rodytiRezultatus()              | 31 |
| 6.11.1.14 rodytiVisusRezultatus()         | 31 |
| 6.11.1.15 skirstytiStudentus()            | 31 |
| 6.11.1.16 testuotiDestruktoriu()          | 31 |
| 6.11.1.17 testuotiFailuNuskaityma()       | 31 |
| 6.11.1.18 vardoTikrinimas()               | 31 |
| 6.12 README.md File Reference             | 32 |
| 6.13 studentClass.h File Reference        | 32 |
| 6.13.1 Function Documentation             | 33 |
| 6.13.1.1 comparePagalEgza()               | 33 |
| 6.13.1.2 comparePagalPavarde()            | 33 |
| 6.13.1.3 generuotiFailus()                | 33 |
| 6.13.1.4 generuotiPazymius()              | 33 |
| 6.13.1.5 generuotiStudentus()             | 33 |
| 6.13.1.6 isvestiDuFailus()                | 33 |
| 6.13.1.7 nuskaitytiFaila()                | 33 |

---

|                                                       |           |
|-------------------------------------------------------|-----------|
| 6.13.1.8 pasirinktiFaila()                            | 33        |
| 6.13.1.9 pasirinktiRikiavima()                        | 34        |
| 6.13.1.10 rikiavimasIrrasymasVargsiukamsIrkietekams() | 34        |
| 6.13.1.11 rikiuotiPagalGalutiniMed()                  | 34        |
| 6.13.1.12 rikiuotiPagalGalutiniVid()                  | 34        |
| 6.13.1.13 rikiuotiPagalPavarde()                      | 34        |
| 6.13.1.14 rikiuotiPagalVarda()                        | 34        |
| 6.13.1.15 rodytiRezultatus()                          | 34        |
| 6.13.1.16 rodytiVisusRezultatus()                     | 34        |
| 6.13.1.17 skirstytiStudentus()                        | 34        |
| 6.13.1.18 testuotiDestruktoriu()                      | 35        |
| 6.13.1.19 testuotiFailuNuskaityma()                   | 35        |
| 6.13.1.20 vardoTikrinimas()                           | 35        |
| 6.14 studentClass.h                                   | 35        |
| 6.15 timer.h File Reference                           | 37        |
| 6.16 timer.h                                          | 37        |
| <b>Index</b>                                          | <b>39</b> |

# Chapter 1

## README

### Programos aprašymas

Programa yra skirta apdoroti studentus ir jų akademinių pasiekimų duomenis, tęsinyje iš objektinis\_↔ programavimas\_2 realizuotas paveldimumas bei Rule of Five, sugeneruota projekto dokumentacija, realizuoti testai naudojantis Catch2.

#### Pradinės programos funkcijos:

1. Įvesti visus studentų duomenis ranka
2. Įvesti studentų vardus, pavardes, atsitiktine tvarka generuoti pažymius
3. Atsitiktinai generuoti studentų vardus, pavardes ir pažymius
4. Nuskaityti duomenis iš failo
5. Atsitiktinai generuoti failą su studentų duomenimis

Programoje taip pat pateiktos **trys strategijos**, skirtos testuoti skirtingų programos veikimo laiko priklausomybę nuo naudojamų konteinerių (*vector*, *list*, *deque*) skirstant studentus į pažangius ir nepažangius.

#### Strategijų aprašymai:

1. Bendro studentų konteinerio (*vector*, *list* ir *deque* tipų) skaidymas (rūšiavimas) į du naujus **to paties tipo** konteinerius: pažangius ir nepažangius.
2. Bendro studentų konteinerio (*vector*, *list* ir *deque*) skaidymas (rūšiavimas) panaudojant tik nepažangių mokinių konteinerį. Tokiu būdu, jei studentas yra nepažangus, jis įkeliamas į nepažangiųjų konteinerį ir ištrinamas iš bendro studentų konteinerio. Atminties atveju ši strategija - efektyvesnė, tačiau dažni trynimai gali būti neefektyvūs tam tikro tipo konteineriams.
3. Bendro studentų konteinerio (*vector*, *list* ir *deque*) skaidymas (rūšiavimas) optimizuojant antrąją strategiją naudojant Standard Template Library (STL)

## Naudojimosi instrukcija

1. Susiinstaliuoti `MAKE`
2. Atsidaryti terminalą
3. Klonuoti programos repozitoriją  
`git clone https://github.com/augustetum/objektinis_programavimas_1.git`
4. Terminale rašyti žemiau esančią komandą, norint paleisti konteinerių tyrimą

```
make runT
```

Terminale rašyti žemiau esančią komandą, norint paleisti 1 strategiją

```
make run1
```

Terminale rašyti žemiau esančią komandą, norint paleisti 2 strategiją

```
make run2
```

Terminale rašyti žemiau esančią komandą, norint paleisti 3 strategiją

```
make run3
```

Terminale rašyti žemiau esančią komandą, norint paleisti pradinę programą

```
make runP
```

Terminale rašyti žemiau esančią komandą, norint paleisti testavimą (v2.0)

```
make test
```

## v2.0

v2.0 sukurta dokumentacija ir atlikti unit testai naudojantis Catch2 testavimo framework. Testai gali būti matomi faile "catchTests.cpp". Pagal žemiau pateiktą kodo iškarpą matoma, jog visi Rule of Five metodai, realizuoti Studento klasėje, veikia korektiškai.

```
Randomness seeded to: 3603547871
=====
All tests passed (7 assertions in 1 test case)
```

## v1.5

v1.5 šakoje pridėta nauja klasė - Žmogus. Studento klasė dabar paveldi Žmogaus klasę, todėl atitinkamai buvo pakeisti ir konstruktoriai / destruktoriai. Nauja Žmogaus bei Studento klasė ištestuota su v1.2 versijoje sukurtu testavimo metodu. Rezultatai parodė, jog nauja klasė veikia su anksčiau implementuotais metodais, t.y. testo rezultatai sutampa su žemiau 1.2 versijoje aprašytais to pačio testo rezultatais.

Žemiau pridedamas testo kodas, leidžiantis geriau suprasti testo rezultatus:

```
cout << "Pasirinkote testuoti klasę: " << endl;
cout << "---- Default konstruktorius ----" << endl;
Studentas s1;
cout << "Egzaminas: " << s1.egzaminas() << endl;
cout << endl;
cout << "---- Vardo, pavardės konstruktorius ----" << endl;
Studentas s2("Jonas", "Jonaitis");
cout << "Vardas: " << s2.vardas() << ", pavardė: " << s2.pavarde() << endl;
cout << endl;
cout << "---- Pilnas konstruktorius ----" << endl;
vector<int> pazymiai = {8, 9, 10, 7, 8};
Studentas s3("Petras", "Petraitis", pazymiai, 9);
cout << "Vardas: " << s3.vardas()
<< ", pavardė: " << s3.pavarde()
<< ", egzaminas: " << s3.egzaminas()
<< ", galutinis vid: " << s3.galutinisVid()
```



```

« ", galutinis med: " « s3.galutinisMed() « endl;
cout « endl;
cout « "---- Copy konstruktorius ----" « endl;
Studentas s4(s3);
cout « "s3 kopija - vardas: " « s4.vardas()
« ", pavarde: " « s4.pavarde()
« ", egzaminas: " « s4 egzaminas()
« ", galutinis vid: " « s4.galutinisVid() « endl;
cout « endl;
s4.setVardas("Antanas");
cout « "Po kopijos pakeitimo - s3.vardas: " « s3.vardas()
« ", s4.vardas: " « s4.vardas() « endl;
cout « endl;
cout « "---- Move konstruktorius ----" « endl;
Studentas s5(std::move(s4));
cout « "Movint'as iš s4 - vardas: " « s5.vardas()
« ", pavarde: " « s5.pavarde()
« ", egzaminas: " « s5 egzaminas() « endl;
cout « endl;
cout « "---- Copy assignment operatorius ----" « endl;
Studentas s6;
s6 = s3;
cout « "s6 po s3 kopijavimo - vardas: " « s6.vardas()
« ", pavarde: " « s6.pavarde() « endl;
cout « endl;
cout « "---- Move assignment operatorius ----" « endl;
Studentas s7;
s7 = std::move(s6);
cout « "s7 po movinimo iš s6 - vardas: " « s7.vardas()
« ", pavarde: " « s7.pavarde() « endl;
cout « endl;
cout « "---- Output'o operatorius ----" « endl;
cout « "s3 naudojant operatorių «: " « s3 « endl;
cout « endl;
cout « "---- Input'o operatorius ----" « endl;
string testInput = "Tomas Tomauskas 5 8 9 10 7 9";
istringstream iss(testInput);
Studentas s8;
iss >> s8;
cout « "s8 po input: " « s8 « endl;
cout « endl;
cout « "---- Destruktorius ----" « endl;
testuotiDestruktoriu();
cout « "---- Žmogaus konstruktoriaus ----" « endl;
Zmogus z("Pranas", "Pranaitis");
cout « z.vardas() « " " « z.pavarde() « endl;

```

Kaip matoma iš žemiau pridėtos ekrano nuotraukos, testo rezultatai nuo v1.2 nepasikeitė:

Tačiau, neleidžiama sukurti žmogaus objekto, dėl jo abstraktumo:

```

augustetumaite@Augustes-Air objektinis_programavimas_2 % make runP
g++ -std=c++17 -I/opt/homebrew/Cellar/catch2/3.8.1/include -c -o programaFunkcijos.o
    programaFunkcijos.cpp
g++ -std=c++17 -I/opt/homebrew/Cellar/catch2/3.8.1/include -c programa.cpp -o programa.o
programa.cpp:321:28: error: variable type 'Zmogus' is an abstract class
    Zmogus z("Pranas", "Pranaitis");
    ^
./humanClass.h:23:22: note: unimplemented pure virtual method 'printInfo' in 'Zmogus'
    virtual void printInfo() const = 0;
    ^
1 error generated.
make: *** [programa.o] Error 1

```

## v1.2

Ankstesnėje v1.2 šakoje implementuoti ir testuoti "Rule of five" metodai bei perdengti įvesties ir išvesties operatoriai. Jų veikimas pritaikytas anksčiau naudotose programos dalyse, tokiose kaip: nuskaitymas iš failo, studentų įvedimas ranka, studentų rezultatų išvedimas.

Žemiau pateikiamos kodo iškarpos, rodančios "Rule of five" bei operatorių perdengimą.

```

//copy konstruktorius
Studentas(const Studentas &s)
: vardas_(s.vardas_),
  pavarde_(s.pavarde_),
  egzaminas_(s egzaminas_),
  galutinisMed_(s.galutinisMed_),
  galutinisVid_(s.galutinisVid_),

```

```

        pazymiuVidurkis_{s.pazymiuVidurkis_},
        pazymiai_{s.pazymiai_}{}

//move konstruktorius
Studentas(Studentas&& s)
: vardas_{std::move(s.vardas_)},
  pavarde_{std::move(s.pavarde_)},
  egzaminas_{s.egzaminas_},
  galutinisMed_{s.galutinisMed_},
  galutinisVid_{s.galutinisVid_},
  pazymiuVidurkis_{s.pazymiuVidurkis_},
  pazymiai_{std::move(s.pazymiai_)}{}

//copy assignment operatorius
Studentas& operator=(const Studentas& s){
    if (this == &s) return *this;

    vardas_ = s.vardas_;
    pavarde_ = s.pavarde_;
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = s.pazymiai_;

    return *this;
}

//move assignment operatorius
Studentas& operator=(Studentas&& s){
    if (this == &s) return *this;

    vardas_ = std::move(s.vardas_);
    pavarde_ = std::move(s.pavarde_);
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = std::move(s.pazymiai_);

    return *this;
}

//destruktorius
~Studentas(){
    vardas_.clear();
    pavarde_.clear();
    pazymiai_.clear();
};

//perdengtas įvesties operatorius (panaikintas vardo ir pavardės tikrinimas dėl failuose esančio
//formatavimo, kur vardas ir pavardė GALI turėti skaičius)
std::istream& operator>>(std::istream& cin, Studentas& s){
    string vardas, pavarde;
    vector<int> pazymiai;
    int egzaminas, pazymys;

    cin >> vardas >> pavarde;

    if (cin.eof()) {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }

    while (cin >> pazymys){
        if (cin.fail()){
            if (cin.eof()){
                break;
            }
            cin.clear();
            cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra skaitinės reikšmės");
        }
        if (pazymys < 1 || pazymys > 10){
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra sveiki skaičiai
            ribose nuo 1 iki 10");
        }
        pazymiai.push_back(pazymys);
    }
    if (!pazymiai.empty()) {
        egzaminas = pazymiai.back();
        pazymiai.pop_back();
    } else {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }
    s = Studentas(vardas, pavarde, pazymiai, egzaminas);
    return cin;
}

```

```

}

//perdengtas išvesties operatorius
std::ostream& operator<(std::ostream& out, const Studentas &s) {
    out << std::left << std::setw(20) << s.pavarde() << std::setw(20) << s.vardas() << std::setw(20) <<
    std::fixed << std::setprecision(2) << s.galutinisVid() << std::setw(20) << std::fixed <<
    std::setprecision(2) << s.galutinisMed() << endl;
    return out;
}

```

v1.2 versijoje sukurta klasės testavimo programa leidžia testuoti visas minėtas funkcijas, gaunami rezultatai:

```

---- Default konstruktorius ----
Egzaminas: 0

---- Vardo, pavardės konstruktorius ----
Vardas: Jonas, pavardė: Jonaitis

---- Pilnas konstruktorius ----
Vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76, galutinis med: 8.6

---- Copy konstruktorius ----
s3 kopija - vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76

Po kopijos pakeitimo - s3.vardas: Petras, s4.vardas: Antanas

---- Move konstruktorius ----
Movint'as iš s4 - vardas: Antanas, pavardė: Petraitis, egzaminas: 9

---- Copy assignment operatorius ----
s6 po s3 kopijavimo - vardas: Petras, pavardė: Petraitis

---- Move assignment operatorius ----
s7 po movinimo iš s6 - vardas: Petras, pavardė: Petraitis

---- Output'o operatorius ----
s3 naudojant operatorių <: Petraitis          Petras          8.76          8.60

---- Input'o operatorius ----
s8 po input: Tomauskas          Tomas          8.52          8.60

---- Destruktorius ----
Destruktorius suveikė

```

## v1.1

Ankstesnė v1.1 šaka skirta palyginti "class" ir "struct" naudojimą talpinant studento duomenis. Tyrimas ir jo rezultatai pateikiami žemiau, v1.1 implementuota "class" lyginama su v1.0 versijoje naudotu "struct".

Tyrimas vykdytas su 3 strategija (žr. v1.0) bei vektoriaus kontaineriu, vykdytos 3 iteracijos.

### "Class" ir "struct" | Tyrimo rezultatai

| Programos veikimo laikas su CLASS | Programos veikimo laikas su STRUCT | Studentų kiekis | Optimizavimo vėliava | .exe failo dydis su CLASS (KB) | .exe failo dydis su STRUCT |
|-----------------------------------|------------------------------------|-----------------|----------------------|--------------------------------|----------------------------|
| 0.434467s                         | 0.418547s                          | 100 000         | -O3                  | 332                            | 329                        |
| 0.453639s                         | 0.428051s                          | 100 000         | -O2                  | 319                            | 315                        |
| 0.453065s                         | 0.427737s                          | 100 000         | -O1                  | 317                            | 331                        |
| 1.54309s                          | 1.32805s                           | 100 000         | -                    | 861                            | 841                        |
| 1.95142s                          | 1.70978s                           | 1 000 000       | -O3                  | 332                            | 329                        |
| 1.9155s                           | 1.73843s                           | 1 000 000       | -O2                  | 319                            | 315                        |
| 1.93125s                          | 1.7828s                            | 1 000 000       | -O1                  | 317                            | 331                        |
| 7.94422s                          | 6.41446s                           | 1 000 000       | -                    | 861                            | 841                        |



## Chapter 2

# Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                     |    |
|---------------------|----|
| Timer . . . . .     | 17 |
| Zmogus . . . . .    | 18 |
| Studentas . . . . . | 13 |



## Chapter 3

# Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                           |    |
|---------------------------|----|
| <a href="#">Studentas</a> | 13 |
| <a href="#">Timer</a>     | 17 |
| <a href="#">Zmogus</a>    | 18 |





# Chapter 4

## File Index

### 4.1 File List

Here is a list of all files with brief descriptions:

|                                                     |    |
|-----------------------------------------------------|----|
| <a href="#">catchTests.cpp</a>                      | 27 |
| <a href="#">humanClass.h</a>                        | 28 |
| <a href="#">programa.cpp</a>                        | 28 |
| <a href="#">programaFunkcijos.cpp</a>               | 29 |
| <a href="#">studentClass.h</a>                      | 32 |
| <a href="#">timer.h</a>                             | 37 |
| <a href="#">1_strategija/konteineriuTyrimas.cpp</a> | 21 |
| <a href="#">1_strategija/strategija1.cpp</a>        | 21 |
| <a href="#">1_strategija/strategija2.cpp</a>        | 22 |
| <a href="#">1_strategija/strategija3.cpp</a>        | 22 |
| <a href="#">1_strategija/strategijos_bibl.h</a>     | 22 |



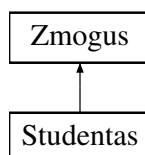
## Chapter 5

# Class Documentation

### 5.1 Studentas Class Reference

```
#include <studentClass.h>
```

Inheritance diagram for Studentas:



#### Public Member Functions

- [Studentas](#) (string var, string pav, vector< int > paz, int egz)
- [Studentas](#) (string var, string pav)
- [Studentas](#) ()
- [Studentas](#) (std::istream &is)
- [Studentas](#) (const [Studentas](#) &s)
- [Studentas](#) ([Studentas](#) &&s)
- [~Studentas](#) () override
- double [galutinisMed](#) () const
- double [galutinisVid](#) () const
- const vector< int > & [pazymiai](#) () const
- int [egzaminas](#) () const
- void [setGalutinisV](#) (double galutVid)
- void [setGalutinisM](#) (double galutMed)
- void [setEgzaminas](#) (int egz)
- void [setPazymiai](#) (vector< int > paz)
- void [skaiciuotiGalutiniSuVid](#) ()
- void [skaiciuotiGalutiniSuMed](#) ()
- [Studentas](#) & [operator=](#) (const [Studentas](#) &s)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&s)
- bool [operator==](#) (const [Studentas](#) &s) const
- bool [operator!=](#) (const [Studentas](#) &s) const
- void [printInfo](#) () const override
- void [clearStud](#) ()
- bool [isClear](#) ()

## Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (string var, string pav)
- std::string [vardas](#) () const
- std::string [pavarde](#) () const
- void [setVardas](#) (string var)
- void [setPavarde](#) (string pav)
- virtual [~Zmogus](#) ()

## Friends

- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)
- std::istream & [operator>>](#) (std::istream &cin, [Studentas](#) &s)

## Additional Inherited Members

## Protected Attributes inherited from [Zmogus](#)

- string [vardas\\_](#)
- string [pavarde\\_](#)

## 5.1.1 Constructor & Destructor Documentation

### 5.1.1.1 [Studentas\(\)](#) [1/6]

```
Studentas::Studentas (
    string var,
    string pav,
    vector< int > paz,
    int egz) [inline]
```

### 5.1.1.2 [Studentas\(\)](#) [2/6]

```
Studentas::Studentas (
    string var,
    string pav) [inline]
```

### 5.1.1.3 [Studentas\(\)](#) [3/6]

```
Studentas::Studentas () [inline]
```

### 5.1.1.4 [Studentas\(\)](#) [4/6]

```
Studentas::Studentas (
    std::istream & is)
```

#### 5.1.1.5 Studentas() [5/6]

```
Studentas::Studentas (  
    const Studentas & s) [inline]
```

#### 5.1.1.6 Studentas() [6/6]

```
Studentas::Studentas (  
    Studentas && s) [inline]
```

#### 5.1.1.7 ~Studentas()

```
Studentas::~~Studentas () [inline], [override]
```

### 5.1.2 Member Function Documentation

#### 5.1.2.1 clearStud()

```
void Studentas::clearStud () [inline]
```

#### 5.1.2.2 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

#### 5.1.2.3 galutinisMed()

```
double Studentas::galutinisMed () const [inline]
```

#### 5.1.2.4 galutinisVid()

```
double Studentas::galutinisVid () const [inline]
```

#### 5.1.2.5 isClear()

```
bool Studentas::isClear () [inline]
```

#### 5.1.2.6 operator"!="()

```
bool Studentas::operator!= (  
    const Studentas & s) const [inline]
```

#### 5.1.2.7 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & s) [inline]
```

#### 5.1.2.8 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && s) [inline]
```

#### 5.1.2.9 operator==(

```
bool Studentas::operator== (
    const Studentas & s) const [inline]
```

#### 5.1.2.10 pazymiai()

```
const vector< int > & Studentas::pazymiai () const [inline]
```

#### 5.1.2.11 printInfo()

```
void Studentas::printInfo () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

#### 5.1.2.12 setEgzaminas()

```
void Studentas::setEgzaminas (
    int egz) [inline]
```

#### 5.1.2.13 setGalutinisM()

```
void Studentas::setGalutinisM (
    double galutMed) [inline]
```

#### 5.1.2.14 setGalutinisV()

```
void Studentas::setGalutinisV (
    double galutVid) [inline]
```

#### 5.1.2.15 setPazymiai()

```
void Studentas::setPazymiai (
    vector< int > paz) [inline]
```

#### 5.1.2.16 skaiciuotiGalutiniSuMed()

```
void Studentas::skaiciuotiGalutiniSuMed ()
```

#### 5.1.2.17 skaiciuotiGalutiniSuVid()

```
void Studentas::skaiciuotiGalutiniSuVid ()
```

### 5.1.3 Friends And Related Symbol Documentation

#### 5.1.3.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Studentas & s) [friend]
```

#### 5.1.3.2 operator>>

```
std::istream & operator>> (  
    std::istream & cin,  
    Studentas & s) [friend]
```

The documentation for this class was generated from the following files:

- [studentClass.h](#)
- [programaFunkcijos.cpp](#)

## 5.2 Timer Class Reference

```
#include <timer.h>
```

### Public Member Functions

- [Timer](#) ()
- void [reset](#) ()
- double [elapsed](#) () const

### 5.2.1 Constructor & Destructor Documentation

#### 5.2.1.1 Timer()

```
Timer::Timer () [inline]
```

## 5.2.2 Member Function Documentation

### 5.2.2.1 elapsed()

```
double Timer::elapsed () const [inline]
```

### 5.2.2.2 reset()

```
void Timer::reset () [inline]
```

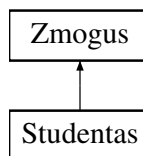
The documentation for this class was generated from the following file:

- [timer.h](#)

## 5.3 Zmogus Class Reference

```
#include <humanClass.h>
```

Inheritance diagram for Zmogus:



### Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (string var, string pav)
- std::string [vardas](#) () const
- std::string [pavarde](#) () const
- void [setVardas](#) (string var)
- void [setPavarde](#) (string pav)
- virtual [~Zmogus](#) ()
- virtual void [printInfo](#) () const =0

### Protected Attributes

- string [vardas\\_](#)
- string [pavarde\\_](#)

## 5.3.1 Constructor & Destructor Documentation

### 5.3.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```



### 5.3.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    string var,
    string pav) [inline]
```

### 5.3.1.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [inline], [virtual]
```

## 5.3.2 Member Function Documentation

### 5.3.2.1 pavarde()

```
std::string Zmogus::pavarde () const [inline]
```

### 5.3.2.2 printInfo()

```
virtual void Zmogus::printInfo () const [pure virtual]
```

Implemented in [Studentas](#).

### 5.3.2.3 setPavarde()

```
void Zmogus::setPavarde (
    string pav) [inline]
```

### 5.3.2.4 setVardas()

```
void Zmogus::setVardas (
    string var) [inline]
```

### 5.3.2.5 vardas()

```
std::string Zmogus::vardas () const [inline]
```

## 5.3.3 Member Data Documentation

### 5.3.3.1 pavarde\_

```
string Zmogus::pavarde_ [protected]
```

### 5.3.3.2 vardas\_

```
string Zmogus::vardas_ [protected]
```

The documentation for this class was generated from the following file:

- [humanClass.h](#)



## Chapter 6

# File Documentation

### 6.1 1\_strategija/konteineriuTyrimas.cpp File Reference

```
#include "../studentClass.h"  
#include "strategijos_bibl.h"
```

#### Functions

- int [main](#) ()

#### 6.1.1 Function Documentation

##### 6.1.1.1 main()

```
int main ()
```

### 6.2 1\_strategija/strategija1.cpp File Reference

```
#include "../studentClass.h"  
#include "strategijos_bibl.h"
```

#### Functions

- int [main](#) ()

#### 6.2.1 Function Documentation

##### 6.2.1.1 main()

```
int main ()
```

## 6.3 1\_strategija/strategija2.cpp File Reference

```
#include "../studentClass.h"  
#include "strategijos_bibl.h"
```

### Functions

- int `main` ()

### 6.3.1 Function Documentation

#### 6.3.1.1 main()

```
int main ()
```

## 6.4 1\_strategija/strategija3.cpp File Reference

```
#include "../studentClass.h"  
#include "strategijos_bibl.h"
```

### Functions

- int `main` ()

### 6.4.1 Function Documentation

#### 6.4.1.1 main()

```
int main ()
```

## 6.5 1\_strategija/strategijos\_bibl.h File Reference

```
#include <list>  
#include "../studentClass.h"  
#include <thread>  
#include <functional>
```

## Functions

- `template<typename Container>`  
`void nuskaitytiFailaT` (string *fail*, Container &studentuSarasas)
- `template<typename Container>`  
`void rikiuotiPagalVardaT` (Container &studentuSarasas)
- `template<typename Container>`  
`void rikiuotiPagalPavardeT` (Container &studentuSarasas)
- `template<typename Container>`  
`void rikiuotiPagalGalutiniMedT` (Container &studentuSarasas)
- `template<typename Container>`  
`void rikiuotiPagalGalutiniVidT` (Container &studentuSarasas)
- `template<typename Container>`  
`void skirstytiStudentusSuTaisPaciaisKonteineriaisT` (Container &studentuSarasas)
- `template<typename Container>`  
`void skirstytiStudentusSuVienuKonteineriuT` (Container &studentuSarasas, int rikiavimas)
- `template<typename Container>`  
`void skirstytiStudentusT` (Container &studentuSarasas)
- `template<typename Container>`  
`void studentuSkirstymas3` (Container &studentuSarasas, int rikiavimas)

## 6.5.1 Function Documentation

### 6.5.1.1 nuskaitytiFailaT()

```
template<typename Container>
void nuskaitytiFailaT (
    string fail,
    Container & studentuSarasas)
```

### 6.5.1.2 rikiuotiPagalGalutiniMedT()

```
template<typename Container>
void rikiuotiPagalGalutiniMedT (
    Container & studentuSarasas)
```

### 6.5.1.3 rikiuotiPagalGalutiniVidT()

```
template<typename Container>
void rikiuotiPagalGalutiniVidT (
    Container & studentuSarasas)
```

### 6.5.1.4 rikiuotiPagalPavardeT()

```
template<typename Container>
void rikiuotiPagalPavardeT (
    Container & studentuSarasas)
```

#### 6.5.1.5 rikiuotiPagalVardaT()

```
template<typename Container>
void rikiuotiPagalVardaT (
    Container & studentuSarasas)
```

#### 6.5.1.6 skirstytiStudentusSuTaisPaciaisKonteineriaisT()

```
template<typename Container>
void skirstytiStudentusSuTaisPaciaisKonteineriaisT (
    Container & studentuSarasas)
```

#### 6.5.1.7 skirstytiStudentusSuVienuKonteineriuT()

```
template<typename Container>
void skirstytiStudentusSuVienuKonteineriuT (
    Container & studentuSarasas,
    int rikiavimas)
```

#### 6.5.1.8 skirstytiStudentusT()

```
template<typename Container>
void skirstytiStudentusT (
    Container & studentuSarasas)
```

#### 6.5.1.9 studentuSkirstymas3()

```
template<typename Container>
void studentuSkirstymas3 (
    Container & studentuSarasas,
    int rikiavimas)
```

## 6.6 strategijos\_bibl.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STRATEGIJOS1_BIBL_H
00002 #define STRATEGIJOS1_BIBL_H
00003
00004 #include <list>
00005 #include "../studentClass.h"
00006 #include <thread>
00007 #include <functional>
00008
00009 using std::deque;
00010 using std::list;
00011
00012 template <typename Container>
00013 void nuskaitytiFailaT(string fail, Container &studentuSarasas){
00014     string eilut;
00015     int pazymys;
00016     std::stringstream buferis;
00017     ifstream failas(fail);
00018     if (!failas.is_open()){
```

```

00019         throw std::runtime_error("Failo nepavyko atidaryti arba jis neegzistuoja šiame
aplankale");
00020     }
00021
00022     buferis « failas.rdbuf();
00023     failas.close();
00024
00025     getline(buferis, eilut);
00026
00027     while(getline(buferis, eilut)){
00028         istreamstring eilute(eilut);
00029         string vardas, pavarde;
00030         vector<int> pazymiai;
00031         int egzaminas;
00032
00033         eilute » vardas » pavarde;
00034
00035         if (eilute.eof()) {
00036             throw "Netinkamas failo formatas: faile nėra pažymių";
00037         }
00038
00039         while(true){
00040             eilute » pazymys;
00041             if (eilute.fail()){
00042                 if (eilute.eof()){
00043                     break;
00044                 }
00045                 eilute.clear();
00046                 eilute.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00047                 throw "Netinkamas failo formatas: pažymiai nėra skaitinės reikšmės";
00048             }
00049             if (pazymys < 1 || pazymys > 10){
00050                 throw "Netinkamas failo formatas: pažymiai nėra sveiki skaičiai ribose nuo 1
iki 10";
00051             }
00052             pazymiai.push_back(pazymys);
00053         }
00054         if (!pazymiai.empty()) {
00055             egzaminas = pazymiai.back();
00056             pazymiai.pop_back();
00057         } else {
00058             throw "Netinkamas failo formatas: faile nėra pažymių";
00059         }
00060         Studentas stud(vardas, pavarde, pazymiai, egzaminas);
00061         studentuSarasas.push_back(stud);
00062         pazymiai.clear();
00063     }
00064 }
00065
00066 template <typename Container>
00067 void rikiuotiPagalVardaT(Container &studentuSarasas){
00068     if constexpr (std::is_same_v<Container, std::list<Studentas>) {
00069         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00070             return a.vardas() < b.vardas();
00071         });
00072     } else {
00073         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
b) {
00074             return a.vardas() < b.vardas();
00075         });
00076     }
00077 }
00078 }
00079
00080 template <typename Container>
00081 void rikiuotiPagalPavardeT(Container &studentuSarasas){
00082     if constexpr (std::is_same_v<Container, std::list<Studentas>) {
00083         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00084             return a.pavarde() < b.pavarde();
00085         });
00086     } else {
00087         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
b) {
00088             return a.pavarde() < b.pavarde();
00089         });
00090     }
00091 }
00092 }
00093
00094 template <typename Container>
00095 void rikiuotiPagalGalutiniMedT(Container &studentuSarasas){
00096     if constexpr (std::is_same_v<Container, std::list<Studentas>) {
00097         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00098             return a.galutinisMed() < b.galutinisMed();
00099         });
00100     } else {
00101         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&

```

```

    b) {
00102         return a.galutinisMed() < b.galutinisMed();
00103     };
00104 }
00105 }
00106 }
00107
00108 template <typename Container>
00109 void rikiuotiPagalGalutiniVidT(Container &studentuSarasas){
00110     if constexpr (std::is_same_v<Container, std::list<Studentas>)> {
00111         studentuSarasas.sort([](const Studentas &a, const Studentas &b) {
00112             return a.galutinisVid() > b.galutinisVid();
00113         });
00114     } else {
00115         sort(studentuSarasas.begin(), studentuSarasas.end(), [](const Studentas& a, const Studentas&
    b) {
00116         return a.galutinisVid() > b.galutinisVid();
00117     });
00118 }
00119 }
00120 }
00121
00122 template <typename Container>
00123 void skirstytiStudentusSuTaisPaciaisKonteineriaisT(Container &studentuSarasas){
00124     Container nepazangus;
00125     Container pazangus;
00126     for(Studentas s : studentuSarasas){
00127         if (s.galutinisVid() < 5){
00128             nepazangus.push_back(s);
00129         } else if (s.galutinisVid() >= 5){
00130             pazangus.push_back(s);
00131         }
00132     }
00133 }
00134
00135 if constexpr (std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
deque<Studentas>)>){
00136     nepazangus.shrink_to_fit();
00137     pazangus.shrink_to_fit();
00138 }
00139
00140 template <typename Container>
00141 void skirstytiStudentusSuVienuKonteineriuT(Container &studentuSarasas, int rikiavimas){
00142     Container nepazangus;
00143     rikiuotiPagalGalutiniVidT(studentuSarasas);
00144     while (studentuSarasas.back().galutinisVid() < 5) {
00145         nepazangus.push_back(studentuSarasas.back());
00146         studentuSarasas.pop_back();
00147     }
00148 }
00149
00150 if constexpr (std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
deque<Studentas>)>){
00151     studentuSarasas.shrink_to_fit();
00152     nepazangus.shrink_to_fit();
00153 }
00154
00155 if (rikiavimas == 1) {
00156     rikiuotiPagalVardaT(studentuSarasas);
00157     rikiuotiPagalVardaT(nepazangus);
00158 } else if (rikiavimas == 2) {
00159     rikiuotiPagalPavardeT(studentuSarasas);
00160     rikiuotiPagalPavardeT(nepazangus);
00161 } else if (rikiavimas == 3) {
00162     rikiuotiPagalGalutiniVidT(studentuSarasas);
00163     rikiuotiPagalGalutiniVidT(nepazangus);
00164 } else if (rikiavimas == 4) {
00165     rikiuotiPagalGalutiniMedT(studentuSarasas);
00166     rikiuotiPagalGalutiniMedT(nepazangus);
00167 }
00168 }
00169 }
00170
00171 template <typename Container>
00172 void skirstytiStudentusT(Container &studentuSarasas){
00173     Container nepazangus;
00174     Container pazangus;
00175     for(Studentas s : studentuSarasas){
00176         if (s.galutinisVid() < 5){
00177             nepazangus.push_back(s);
00178         } else if (s.galutinisVid() >= 5){
00179             pazangus.push_back(s);
00180         }
00181     }
00182 }
00183
00184 if constexpr (std::is_same_v<Container, vector<Studentas> || std::is_same_v<Container,
deque<Studentas>)>){

```



```

00184         nepazangus.shrink_to_fit();
00185         pazangus.shrink_to_fit();
00186     }
00187     studentuSarasas.clear();
00188 }
00189
00190 template <typename Container>
00191 void studentuSkirstymas3(Container &studentuSarasas, int rikiavimas){
00192     auto it = std::partition(studentuSarasas.begin(), studentuSarasas.end(), [](const auto &studentas)
00193     { return studentas.galutinisVid() >= 5; });
00194     Container nepazangus;
00195     nepazangus.assign(it, studentuSarasas.end());
00196     studentuSarasas.resize(std::distance(studentuSarasas.begin(), it));
00197     std::thread t1, t2;
00198
00199     if (rikiavimas == 1) {
00200         std::thread t1([&studentuSarasas]() { rikiuotiPagalVardaT(studentuSarasas); });
00201         std::thread t2([&nepazangus]() { rikiuotiPagalVardaT(nepazangus); });
00202         t1.join();
00203         t2.join();
00204     } else if (rikiavimas == 2) {
00205         std::thread t1([&studentuSarasas]() { rikiuotiPagalPavardeT(studentuSarasas); });
00206         std::thread t2([&nepazangus]() { rikiuotiPagalPavardeT(nepazangus); });
00207         t1.join();
00208         t2.join();
00209     } else if (rikiavimas == 3) {
00210         std::thread t1([&studentuSarasas]() { rikiuotiPagalGalutiniVidT(studentuSarasas); });
00211         std::thread t2([&nepazangus]() { rikiuotiPagalGalutiniVidT(nepazangus); });
00212         t1.join();
00213         t2.join();
00214     } else if (rikiavimas == 4) {
00215         std::thread t1([&studentuSarasas]() { rikiuotiPagalGalutiniMedT(studentuSarasas); });
00216         std::thread t2([&nepazangus]() { rikiuotiPagalGalutiniMedT(nepazangus); });
00217         t1.join();
00218         t2.join();
00219     }
00220
00221     if (t1.joinable()) t1.join();
00222     if (t2.joinable()) t2.join();
00223 }
00224 #endif
00225

```

## 6.7 catchTests.cpp File Reference

```

#include <catch2/catch_all.hpp>
#include "studentClass.h"

```

### Macros

- `#define CATCH_CONFIG_MAIN`

### Functions

- `TEST_CASE` ("Studento Rule of Five testai", "[studentas]")

## 6.7.1 Macro Definition Documentation

### 6.7.1.1 CATCH\_CONFIG\_MAIN

```
#define CATCH_CONFIG_MAIN
```

## 6.7.2 Function Documentation

### 6.7.2.1 TEST\_CASE()

```
TEST_CASE (
    "Studento Rule of Five testai" ,
    "" [studentas])
```

## 6.8 humanClass.h File Reference

```
#include <iostream>
```

### Classes

- class [Zmogus](#)

## 6.9 humanClass.h

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 using std::string;
00003
00004 class Zmogus {
00005     protected:
00006         string vardas_;
00007         string pavarde_;
00008     public:
00009         Zmogus() {}
00010         Zmogus(string var, string pav) : vardas_{var}, pavarde_{pav} {}
00011         //get'eriai
00012         inline std::string vardas() const { return vardas_; }
00013         inline std::string pavarde() const { return pavarde_; }
00014         //set'eriai
00015         inline void setVardas(string var) { vardas_ = var; }
00016         inline void setPavarde(string pav) { pavarde_ = pav; }
00017
00018         virtual ~Zmogus() {
00019             vardas_.clear();
00020             pavarde_.clear();
00021         }
00022
00023         virtual void printInfo() const = 0;
00024 };
```

## 6.10 programa.cpp File Reference

```
#include "studentClass.h"
```

### Functions

- int [main](#) ()

## 6.10.1 Function Documentation

### 6.10.1.1 main()

```
int main ()
```

## 6.11 programaFunkcijos.cpp File Reference

```
#include "studentClass.h"
```

### Functions

- void [rodytiRezultatus](#) (vector< [Studentas](#) > studentuSarasas)
- void [rodytiVisusRezultatus](#) (vector< [Studentas](#) > studentuSarasas)
- void [generuotiPazymius](#) (vector< [Studentas](#) > &studentuSarasas)
- void [generuotiStudentus](#) (vector< [Studentas](#) > &studentuSarasas)
- void [generuotiFailus](#) (int studentuSkaicius)
- string [pasirinktiFaila](#) ()
- void [nuskaitytiFaila](#) (string fail, vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalVarda](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalPavarde](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalGalutiniMed](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalGalutiniVid](#) (vector< [Studentas](#) > &studentuSarasas)
- void [testuotiFailuNuskaityma](#) (vector< [Studentas](#) > studentuSarasas, int kartai)
- bool [vardoTikrinimas](#) (const string &vard)
- void [skirstytiStudentus](#) (vector< [Studentas](#) > &studentuSarasas)
- void [investiDuFailus](#) (vector< [Studentas](#) > grupe1, vector< [Studentas](#) > grupe2)
- std::istream & [operator>>](#) (std::istream &cin, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &s)
- void [testuotiDestruktoriu](#) ()

## 6.11.1 Function Documentation

### 6.11.1.1 generuotiFailus()

```
void generuotiFailus (  
    int studentuSkaicius)
```

### 6.11.1.2 generuotiPazymius()

```
void generuotiPazymius (  
    vector< Studentas > & studentuSarasas)
```

### 6.11.1.3 generuotiStudentus()

```
void generuotiStudentus (  
    vector< Studentas > & studentuSarasas)
```

#### 6.11.1.4 isvestiDuFailus()

```
void isvestiDuFailus (
    vector< Studentas > grupe1,
    vector< Studentas > grupe2)
```

#### 6.11.1.5 nuskaitytiFaila()

```
void nuskaitytiFaila (
    string fail,
    vector< Studentas > & studentuSarasas)
```

#### 6.11.1.6 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s)
```

#### 6.11.1.7 operator>>()

```
std::istream & operator>> (
    std::istream & cin,
    Studentas & s)
```

#### 6.11.1.8 pasirinktiFaila()

```
string pasirinktiFaila ()
```

#### 6.11.1.9 rikiuotiPagalGalutiniMed()

```
void rikiuotiPagalGalutiniMed (
    vector< Studentas > & studentuSarasas)
```

#### 6.11.1.10 rikiuotiPagalGalutiniVid()

```
void rikiuotiPagalGalutiniVid (
    vector< Studentas > & studentuSarasas)
```

#### 6.11.1.11 rikiuotiPagalPavarde()

```
void rikiuotiPagalPavarde (
    vector< Studentas > & studentuSarasas)
```

**6.11.1.12 rikiuotiPagalVarda()**

```
void rikiuotiPagalVarda (  
    vector< Studentas > & studentuSarasas)
```

**6.11.1.13 rodytiRezultatus()**

```
void rodytiRezultatus (  
    vector< Studentas > studentuSarasas)
```

**6.11.1.14 rodytiVisusRezultatus()**

```
void rodytiVisusRezultatus (  
    vector< Studentas > studentuSarasas)
```

**6.11.1.15 skirstytiStudentus()**

```
void skirstytiStudentus (  
    vector< Studentas > & studentuSarasas)
```

**6.11.1.16 testuotiDestruktoriu()**

```
void testuotiDestruktoriu ()
```

**6.11.1.17 testuotiFailuNuskaityma()**

```
void testuotiFailuNuskaityma (  
    vector< Studentas > studentuSarasas,  
    int kartai)
```

**6.11.1.18 vardoTikrinimas()**

```
bool vardoTikrinimas (  
    const string & vard)
```

## 6.12 README.md File Reference

## 6.13 studentClass.h File Reference

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <fstream>
#include <string>
#include <numeric>
#include <ctime>
#include <sstream>
#include <chrono>
#include <unordered_set>
#include <limits>
#include <ios>
#include "timer.h"
#include "humanClass.h"
```

### Classes

- class [Studentas](#)

### Functions

- bool [comparePagalPavarde](#) (const [Studentas](#) &, const [Studentas](#) &)
- bool [comparePagalEgza](#) (const [Studentas](#) &, const [Studentas](#) &)
- void [rodytiRezultatus](#) (vector< [Studentas](#) > studentuSarasas)
- void [generuotiPazymius](#) (vector< [Studentas](#) > &studentuSarasas)
- void [generuotiStudentus](#) (vector< [Studentas](#) > &studentuSarasas)
- void [generuotiFailus](#) (int studentuSkaicius)
- string [pasirinktiFaila](#) ()
- void [nuskaitytiFaila](#) (string fail, vector< [Studentas](#) > &studentuSarasas)
- void [rodytiVisusRezultatus](#) (vector< [Studentas](#) > studentuSarasas)
- void [testuotiFailuNuskaityma](#) (vector< [Studentas](#) > studentuSarasas, int kartai)
- void [rikiuotiPagalVarda](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalPavarde](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalGalutiniMed](#) (vector< [Studentas](#) > &studentuSarasas)
- void [rikiuotiPagalGalutiniVid](#) (vector< [Studentas](#) > &studentuSarasas)
- void [pasirinktiRikiavima](#) (vector< [Studentas](#) > studentuSarasas)
- void [skirstytiStudentus](#) (vector< [Studentas](#) > &studentuSarasas)
- bool [vardoTikrinimas](#) (const string &vard)
- void [investiDuFailus](#) (vector< [Studentas](#) > grupe1, vector< [Studentas](#) > grupe2)
- void [rikiavimasIrlrasymasVargsiukamsIrkietekams](#) (vector< [Studentas](#) > vargsiukai, vector< [Studentas](#) > kietekai)
- void [testuotiDestruktoriu](#) ()

## 6.13.1 Function Documentation

### 6.13.1.1 comparePagalEgza()

```
bool comparePagalEgza (
    const Studentas & ,
    const Studentas & )
```

### 6.13.1.2 comparePagalPavarde()

```
bool comparePagalPavarde (
    const Studentas & ,
    const Studentas & )
```

### 6.13.1.3 generuotiFailus()

```
void generuotiFailus (
    int studentuSkaicius)
```

### 6.13.1.4 generuotiPazymius()

```
void generuotiPazymius (
    vector< Studentas > & studentuSarajas)
```

### 6.13.1.5 generuotiStudentus()

```
void generuotiStudentus (
    vector< Studentas > & studentuSarajas)
```

### 6.13.1.6 isvestiDuFailus()

```
void isvestiDuFailus (
    vector< Studentas > grupe1,
    vector< Studentas > grupe2)
```

### 6.13.1.7 nuskaitytiFaila()

```
void nuskaitytiFaila (
    string fail,
    vector< Studentas > & studentuSarajas)
```

### 6.13.1.8 pasirinktiFaila()

```
string pasirinktiFaila ()
```

#### 6.13.1.9 pasirinktiRikiavima()

```
void pasirinktiRikiavima (
    vector< Studentas > studentuSarasas)
```

#### 6.13.1.10 rikiavimasIrIrasymasVargsiukamsIrKietekams()

```
void rikiavimasIrIrasymasVargsiukamsIrKietekams (
    vector< Studentas > vargsiukai,
    vector< Studentas > kietekai)
```

#### 6.13.1.11 rikiuotiPagalGalutiniMed()

```
void rikiuotiPagalGalutiniMed (
    vector< Studentas > & studentuSarasas)
```

#### 6.13.1.12 rikiuotiPagalGalutiniVid()

```
void rikiuotiPagalGalutiniVid (
    vector< Studentas > & studentuSarasas)
```

#### 6.13.1.13 rikiuotiPagalPavarde()

```
void rikiuotiPagalPavarde (
    vector< Studentas > & studentuSarasas)
```

#### 6.13.1.14 rikiuotiPagalVarda()

```
void rikiuotiPagalVarda (
    vector< Studentas > & studentuSarasas)
```

#### 6.13.1.15 rodytiRezultatus()

```
void rodytiRezultatus (
    vector< Studentas > studentuSarasas)
```

#### 6.13.1.16 rodytiVisusRezultatus()

```
void rodytiVisusRezultatus (
    vector< Studentas > studentuSarasas)
```

#### 6.13.1.17 skirstytiStudentus()

```
void skirstytiStudentus (
    vector< Studentas > & studentuSarasas)
```



**6.13.1.18 testuotiDestruktoriu()**

```
void testuotiDestruktoriu ()
```

**6.13.1.19 testuotiFailuNuskaityma()**

```
void testuotiFailuNuskaityma (
    vector< Studentas > studentuSarasas,
    int kartai)
```

**6.13.1.20 vardoTikrinimas()**

```
bool vardoTikrinimas (
    const string & vard)
```

**6.14 studentClass.h**

[Go to the documentation of this file.](#)

```
00001 #ifndef MANO_LIB_H
00002 #define MANO_LIB_H
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <vector>
00007 #include <fstream>
00008 #include <string>
00009 #include <numeric>
00010 #include <ctime>
00011 #include <sstream>
00012 #include <chrono>
00013 #include <unordered_set>
00014 #include <limits>
00015 #include <ios>
00016 #include "timer.h"
00017 #include "humanClass.h"
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::string;
00022 using std::vector;
00023 using std::endl;
00024 using std::accumulate;
00025 using std::ifstream;
00026 using std::istringstream;
00027
00028 class Studentas : public Zmogus {
00029     private:
00030         int egzaminas_;
00031         double galutinisMed_;
00032         double galutinisVid_;
00033         double pazymiuVidurkis_;
00034         vector<int> pazymiai_;
00035
00036     public:
00037         Studentas(string var, string pav, vector<int> paz, int egz) :
00038             Zmogus(var, pav), pazymiai_{paz}, egzaminas_{egz} {
00039             skaiciuotiGalutiniSuMed();
00040             skaiciuotiGalutiniSuVid();
00041         }
00042
00043         Studentas(string var, string pav) : Zmogus(var, pav), egzaminas_(0) {
00044         }
00045         Studentas() : Zmogus(), egzaminas_(0) {}
00046         Studentas(std::istream& is);
00047
00048         //copy konstruktorius
00049         Studentas(const Studentas &s)
00050             : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
```

```

00051         galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
pazymiuVidurkis_{s.pazymiuVidurkis_},
00052         pazymiai_{s.pazymiai_}{}
00053
00054         //move konstruktorius
00055         Studentas(Studentas&& s)
00056         : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
00057         galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
pazymiuVidurkis_{s.pazymiuVidurkis_},
00058         pazymiai_{std::move(s.pazymiai_)}{
00059             s.clearStud();
00060         }
00061
00062         //destruktorius
00063         ~Studentas() override{
00064             clearStud();
00065         };
00066
00067         //get'eriai
00068         inline double galutinisMed() const { return galutinisMed_; }
00069         inline double galutinisVid() const { return galutinisVid_; }
00070         inline const vector<int>& pazymiai() const { return pazymiai_; }
00071         inline int egzaminas() const { return egzaminas_; }
00072
00073         //set'eriai
00074         inline void setGalutinisV(double galutVid) { galutinisVid_ = galutVid; }
00075         inline void setGalutinisM(double galutMed) { galutinisMed_ = galutMed; }
00076         inline void setEgzaminas(int egz) { egzaminas_ = egz; }
00077         inline void setPazymiai(vector<int> paz) { pazymiai_ = paz; }
00078
00079         //member funkcijos
00080         void skaiciuotiGalutiniSuVid();
00081         void skaiciuotiGalutiniSuMed();
00082
00083         //copy assignment operatorius
00084         Studentas& operator=(const Studentas& s){
00085             if (this == &s) return *this;
00086
00087             setVardas(s.vardas());
00088             setPavarde(s.pavarde());
00089             egzaminas_ = s.egzaminas_;
00090             galutinisMed_ = s.galutinisMed_;
00091             galutinisVid_ = s.galutinisVid_;
00092             pazymiuVidurkis_ = s.pazymiuVidurkis_;
00093             pazymiai_ = s.pazymiai_;
00094
00095             return *this;
00096         }
00097
00098         //move assignment operatorius
00099         Studentas& operator =(Studentas&& s){
00100             if (this == &s) return *this;
00101
00102             setVardas(s.vardas());
00103             setPavarde(s.pavarde());
00104             egzaminas_ = s.egzaminas_;
00105             galutinisMed_ = s.galutinisMed_;
00106             galutinisVid_ = s.galutinisVid_;
00107             pazymiuVidurkis_ = s.pazymiuVidurkis_;
00108             pazymiai_ = std::move(s.pazymiai_);
00109             s.clearStud();
00110
00111             return *this;
00112         }
00113
00114         //palyginimo operatorius
00115         bool operator==(const Studentas& s) const{
00116             if(vardas_ == s.vardas_ && pavarde_ == s.pavarde_ && egzaminas_ == s.egzaminas_
&& pazymiai_ == s.pazymiai_){
00117                 return true;
00118             } else {
00119                 return false;
00120             }
00121         }
00122
00123         bool operator!=(const Studentas& s) const{
00124             if(vardas_ != s.vardas_ && pavarde_ != s.pavarde_ && egzaminas_ != s.egzaminas_
&& pazymiai_ != s.pazymiai_){
00125                 return true;
00126             } else {
00127                 return false;
00128             }
00129         }
00130
00131     }
00132
00133     //output operatorius
00134     friend std::ostream& operator<<(std::ostream& out, const Studentas &s);
00135

```

```

00136         //input operatorius
00137         friend std::istream& operator>(std::istream& cin, Studentas& s);
00138
00139         void printInfo() const override {
00140             cout << "Vardas: " << vardas() << ", Pavarde: " << pavarde() << endl;
00141         }
00142
00143         void clearStud () {
00144             this->vardas_.clear();
00145             this->pavarde_.clear();
00146             egzaminas_ = 0;
00147             galutinisMed_ = 0;
00148             galutinisVid_ = 0;
00149             pazymiuVidurkis_ = 0;
00150             pazymiai_.clear();
00151         }
00152
00153         bool isClear(){
00154             if(this->vardas_.empty() && this -> pavarde_.empty() && pazymiai_.empty() && egzaminas_ ==
00155             0 && galutinisMed_ == 0 && galutinisVid_ == 0 &&
00156                 pazymiuVidurkis_ == 0) {
00157                 return true;
00158             } else {
00159                 return false;
00160             }
00161         };
00162
00163         bool comparePagalPavarde(const Studentas&, const Studentas&);
00164         bool comparePagalEgza(const Studentas&, const Studentas&);
00165
00166         void rodytiRezultatus(vector<Studentas> studentuSarasas);
00167         void generuotiPazymius(vector<Studentas> &studentuSarasas);
00168         void generuotiStudentus(vector<Studentas> &studentuSarasas);
00169         void generuotiFailus(int studentuSkaicius);
00170         string pasirinktiFaila();
00171         void nuskaitytiFaila(string fail, vector<Studentas> &studentuSarasas);
00172         void rodytiVisusRezultatus(vector<Studentas> studentuSarasas);
00173         void testuotiFailuNuskaityma(vector<Studentas> studentuSarasas, int kartai);
00174
00175         //rikiavimo funkcijos
00176         void rikiuotiPagalVarda(vector<Studentas> &studentuSarasas);
00177         void rikiuotiPagalPavarde(vector<Studentas> &studentuSarasas);
00178         void rikiuotiPagalGalutiniMed(vector<Studentas> &studentuSarasas);
00179         void rikiuotiPagalGalutiniVid(vector<Studentas> &studentuSarasas);
00180
00181         void pasirinktiRikiavima(vector<Studentas> studentuSarasas);
00182         void skirstytiStudentus(vector<Studentas> &studentuSarasas);
00183
00184         bool vardoTikrinimas(const string &vard);
00185         void isvestiDuFailus(vector<Studentas> grupel, vector<Studentas> grupe2);
00186         void rikiavimasIrIrasymasVargsiukamsIrKietekams(vector<Studentas> vargsiukai, vector<Studentas>
00187             kietekai);
00187         void testuotiDestruktoriu();
00188
00189     #endif

```

## 6.15 timer.h File Reference

```
#include <chrono>
```

### Classes

- class [Timer](#)

## 6.16 timer.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TIMER_H
```

```
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00006 class Timer {
00007     private:
00008         std::chrono::time_point<std::chrono::high_resolution_clock> start;
00009     public:
00010         Timer() : start{std::chrono::high_resolution_clock::now()} {}
00011         void reset() {
00012             start = std::chrono::high_resolution_clock::now();
00013         }
00014         double elapsed() const {return
std::chrono::duration<double>(std::chrono::high_resolution_clock::now() - start).count();
00015         }
00016 };
00017
00018 #endif
```

# Index

- ~Studentas
  - Studentas, [15](#)
- ~Zmogus
  - Zmogus, [19](#)
- 1\_strategija/konteineriuTyrimas.cpp, [21](#)
- 1\_strategija/strategija1.cpp, [21](#)
- 1\_strategija/strategija2.cpp, [22](#)
- 1\_strategija/strategija3.cpp, [22](#)
- 1\_strategija/strategijos\_bibl.h, [22](#), [24](#)
- CATCH\_CONFIG\_MAIN
  - catchTests.cpp, [27](#)
- catchTests.cpp, [27](#)
  - CATCH\_CONFIG\_MAIN, [27](#)
  - TEST\_CASE, [28](#)
- clearStud
  - Studentas, [15](#)
- comparePagalEgza
  - studentClass.h, [33](#)
- comparePagalPavarde
  - studentClass.h, [33](#)
- egzaminas
  - Studentas, [15](#)
- elapsed
  - Timer, [18](#)
- galutinisMed
  - Studentas, [15](#)
- galutinisVid
  - Studentas, [15](#)
- generuotiFailus
  - programaFunkcijos.cpp, [29](#)
  - studentClass.h, [33](#)
- generuotiPazymius
  - programaFunkcijos.cpp, [29](#)
  - studentClass.h, [33](#)
- generuotiStudentus
  - programaFunkcijos.cpp, [29](#)
  - studentClass.h, [33](#)
- humanClass.h, [28](#)
- isClear
  - Studentas, [15](#)
- isvestiDuFailus
  - programaFunkcijos.cpp, [29](#)
  - studentClass.h, [33](#)
- konteineriuTyrimas.cpp
  - main, [21](#)
- main
  - konteineriuTyrimas.cpp, [21](#)
  - programa.cpp, [29](#)
  - strategija1.cpp, [21](#)
  - strategija2.cpp, [22](#)
  - strategija3.cpp, [22](#)
- nuskaitytiFaila
  - programaFunkcijos.cpp, [30](#)
  - studentClass.h, [33](#)
- nuskaitytiFailaT
  - strategijos\_bibl.h, [23](#)
- operator!=
  - Studentas, [15](#)
- operator<<
  - programaFunkcijos.cpp, [30](#)
  - Studentas, [17](#)
- operator>>
  - programaFunkcijos.cpp, [30](#)
  - Studentas, [17](#)
- operator=
  - Studentas, [15](#), [16](#)
- operator==
  - Studentas, [16](#)
- pasirinktiFaila
  - programaFunkcijos.cpp, [30](#)
  - studentClass.h, [33](#)
- pasirinktiRikiavima
  - studentClass.h, [33](#)
- pavarde
  - Zmogus, [19](#)
- pavarde\_
  - Zmogus, [19](#)
- pazymiai
  - Studentas, [16](#)
- printInfo
  - Studentas, [16](#)
  - Zmogus, [19](#)
- programa.cpp, [28](#)
  - main, [29](#)
- programaFunkcijos.cpp, [29](#)
  - generuotiFailus, [29](#)
  - generuotiPazymius, [29](#)
  - generuotiStudentus, [29](#)
  - isvestiDuFailus, [29](#)
  - nuskaitytiFaila, [30](#)
  - operator<<, [30](#)
  - operator>>, [30](#)

- pasirinktiFaila, 30
- rikiuotiPagalGalutiniMed, 30
- rikiuotiPagalGalutiniVid, 30
- rikiuotiPagalPavarde, 30
- rikiuotiPagalVarda, 30
- rodytiRezultatus, 31
- rodytiVisusRezultatus, 31
- skirstytiStudentus, 31
- testuotiDestruktoriu, 31
- testuotiFailuNuskaityma, 31
- vardoTikrinimas, 31
- README, 1
- README.md, 32
- reset
  - Timer, 18
- rikiavimasIrrasymasVargsiukamsIrkietekams
  - studentClass.h, 34
- rikiuotiPagalGalutiniMed
  - programaFunkcijos.cpp, 30
  - studentClass.h, 34
- rikiuotiPagalGalutiniMedT
  - strategijos\_bibl.h, 23
- rikiuotiPagalGalutiniVid
  - programaFunkcijos.cpp, 30
  - studentClass.h, 34
- rikiuotiPagalGalutiniVidT
  - strategijos\_bibl.h, 23
- rikiuotiPagalPavarde
  - programaFunkcijos.cpp, 30
  - studentClass.h, 34
- rikiuotiPagalPavardeT
  - strategijos\_bibl.h, 23
- rikiuotiPagalVarda
  - programaFunkcijos.cpp, 30
  - studentClass.h, 34
- rikiuotiPagalVardaT
  - strategijos\_bibl.h, 23
- rodytiRezultatus
  - programaFunkcijos.cpp, 31
  - studentClass.h, 34
- rodytiVisusRezultatus
  - programaFunkcijos.cpp, 31
  - studentClass.h, 34
- setEgzaminas
  - Studentas, 16
- setGalutinisM
  - Studentas, 16
- setGalutinisV
  - Studentas, 16
- setPavarde
  - Zmogus, 19
- setPazymiai
  - Studentas, 16
- setVardas
  - Zmogus, 19
- skaiciuotiGalutiniSuMed
  - Studentas, 16
- skaiciuotiGalutiniSuVid
  - Studentas, 17
- skirstytiStudentus
  - programaFunkcijos.cpp, 31
  - studentClass.h, 34
- skirstytiStudentusSuTaisPaciaisKonteineriaisT
  - strategijos\_bibl.h, 24
- skirstytiStudentusSuVienuKonteineriuT
  - strategijos\_bibl.h, 24
- skirstytiStudentusT
  - strategijos\_bibl.h, 24
- strategija1.cpp
  - main, 21
- strategija2.cpp
  - main, 22
- strategija3.cpp
  - main, 22
- strategijos\_bibl.h
  - nuskaitytiFailaT, 23
  - rikiuotiPagalGalutiniMedT, 23
  - rikiuotiPagalGalutiniVidT, 23
  - rikiuotiPagalPavardeT, 23
  - rikiuotiPagalVardaT, 23
  - skirstytiStudentusSuTaisPaciaisKonteineriaisT, 24
  - skirstytiStudentusSuVienuKonteineriuT, 24
  - skirstytiStudentusT, 24
  - studentuSkirstymas3, 24
- Studentas, 13
  - ~Studentas, 15
  - clearStud, 15
  - egzaminas, 15
  - galutinisMed, 15
  - galutinisVid, 15
  - isClear, 15
  - operator!=, 15
  - operator<<, 17
  - operator>>, 17
  - operator=, 15, 16
  - operator==, 16
  - pazymiai, 16
  - printlnInfo, 16
  - setEgzaminas, 16
  - setGalutinisM, 16
  - setGalutinisV, 16
  - setPazymiai, 16
  - skaiciuotiGalutiniSuMed, 16
  - skaiciuotiGalutiniSuVid, 17
  - Studentas, 14, 15
- studentClass.h, 32
  - comparePagalEgza, 33
  - comparePagalPavarde, 33
  - generuotiFailus, 33
  - generuotiPazymius, 33
  - generuotiStudentus, 33
  - isvestiDuFailus, 33
  - nuskaitytiFaila, 33
  - pasirinktiFaila, 33
  - pasirinktiRikiavima, 33

- rikiavimasIrrasymasVargsiukamsIrkietekams, [34](#)
- rikiuotiPagalGalutiniMed, [34](#)
- rikiuotiPagalGalutiniVid, [34](#)
- rikiuotiPagalPavarde, [34](#)
- rikiuotiPagalVarda, [34](#)
- rodytiRezultatus, [34](#)
- rodytiVisusRezultatus, [34](#)
- skirstytiStudentus, [34](#)
- testuotiDestruktoriu, [34](#)
- testuotiFailuNuskaityma, [35](#)
- vardoTikrinimas, [35](#)
- studentuSkirstymas3
- strategijos\_bibl.h, [24](#)

## TEST\_CASE

- catchTests.cpp, [28](#)
- testuotiDestruktoriu
  - programaFunkcijos.cpp, [31](#)
  - studentClass.h, [34](#)
- testuotiFailuNuskaityma
  - programaFunkcijos.cpp, [31](#)
  - studentClass.h, [35](#)
- Timer, [17](#)
  - elapsed, [18](#)
  - reset, [18](#)
  - Timer, [17](#)
- timer.h, [37](#)

- vardas
  - Zmogus, [19](#)
- vardas\_
  - Zmogus, [19](#)
- vardoTikrinimas
  - programaFunkcijos.cpp, [31](#)
  - studentClass.h, [35](#)

- Zmogus, [18](#)
  - ~Zmogus, [19](#)
  - pavarde, [19](#)
  - pavarde\_, [19](#)
  - printInfo, [19](#)
  - setPavarde, [19](#)
  - setVardas, [19](#)
  - vardas, [19](#)
  - vardas\_, [19](#)
  - Zmogus, [18](#)